

1. CPU Registers (Fastest Storage)

Registers are the fastest and smallest storage units located inside the processor.

- **General Purpose Registers:** Used for data and intermediate results.
 - **AX:** Accumulator.
 - **CX (Counter):** Specifically used for **Looping**.
 - **BX & DX:** Base and Data registers.
- **Special Purpose Registers:**
 - **PC (Program Counter):** Keeps track of the address of the **next instruction** to be executed.
 - **IR (Instruction Register):** Holds the instruction currently being executed (decoded).
 - **MAR & MBR:** Used to interface with memory.

2. Cache Memory Basics

- **Definition:** A small, fast memory placed between the CPU and Main Memory to bridge the speed gap.
- **Operation Principles:**
 - **Cache Hit:** When requested data is found in the Cache.
 - **Cache Miss:** When data is not found and must be fetched from Main Memory.
 - **Performance Note:** L1 Cache is faster than L3 Cache.

3. Performance Metric: AMAT

The **Average Memory Access Time (AMAT)** is a critical formula for exams:

- **Hit Time:** Time to deliver data from cache to CPU (e.g., 5 ns).
- **Miss Penalty:** Time to copy data from RAM to Cache (e.g., 100 ns).
- **Example:** If Hit time = 5ns, Miss rate = 10%, Penalty = 100ns $AMAT = 15ns$.

4. Cache Mapping Techniques (High Exam Importance)

This defines how a block of data from Main Memory is placed into the Cache lines.

A. Direct Mapping

- **Mechanism:** Each block from main memory maps to exactly **one specific line** in the cache.
- **Formula:** .

- **Pros/Cons:** It is the **simplest** and cheapest to implement but has **low utilization** and high conflict rates.

- **Address Split:** Tag, Index, Offset.

B. Fully Associative Mapping

- **Mechanism:** Data can be stored in **any** unused cache block.

- **Pros/Cons:** It has the highest utilization (no conflicts) but is the **most expensive** to implement because it requires complex circuitry to compare all tags simultaneously.

- **Address Split:** Tag, Offset (No Index needed).

C. Set Associative Mapping

- **Mechanism:** A compromise between Direct and Fully Associative. Cache is divided into **Sets**.

- **Rule:** A block maps to a specific **Set**, but can be placed in **any line** within that Set.

- **Address Split:** Tag, Set Index, Offset.

5. Replacement Policies

When the cache is full and a miss occurs, a block must be replaced.

- **LRU (Least Recently Used):** Replaces the block that has not been used for the longest time. This is the most effective policy.

- **FIFO (First In First Out):** Replaces the oldest block added, regardless of how recently it was used.

- **Random:** Selects a victim block at random.

6. Important Exam Calculations (From Source &)

You will likely face a problem asking to calculate the number of bits for **Tag, Index, and Offset**.

How to solve:

1. **Block Offset:** Depends on **Block Size**.

- *Example:* If Block Size = bytes **Offset = 10 bits**.

2. **Cache Index (Line/Set):** Depends on the number of lines or sets.

- .

- *Example:* Cache = 512 KB, Block = 1 KB 512 lines ().

- **Index = 9 bits.**

3. **Tag Bits:** The remainder of the address.

- .

- *Example:* If Physical Address = 26 bits Tag = bits.

Summary of Key MCQs:

- **CX** is the loop register.
- **Direct Mapping** maps a block into **one specific line**.
- **Fully Associative** is expensive.
- **Latency** is the time from request to first bit access.
- **Cycle Time** is usually longer than access time (Latency).